ZIDIO DEVELOPMENT · PROJECT KEYSTONE

Field Service Management Platform

High-Level Design (HLD) & Low-Level Design (LLD)

Client: Meridian Facilities Management · Spring Boot 3 (Java 21) · React + TypeScript · PostgreSQL · v1.0

PART A — HIGH-LEVEL DESIGN (HLD)

System boundaries, layers, data flow, technology choices — the "what" and "why"

System Architecture

HLD-ARCH

TIER 1 — ACTORS

Dispatcher | Technician | Manager / Admin | Customer

HTTPS + Bearer JWT ▾

TIER 2 — React 18 + TypeScript SPA

Dashboard | Work-order Board | Technician View | Customer Portal

REST / JSON ▾

TIER 3 — Spring Security + JWT (stateless)

JwtAuthFilter | @PreAuthorize | BCrypt

Validated principal ▾

TIER 4 — Spring Boot Application Layer

Controllers | Services | DTOs | Scheduler

Spring Data JPA ▾

TIER 5 — Repositories (role-scoped queries)

JDBC ▾

TIER 6 — PostgreSQL (Flyway-managed)

ARCHITECTURAL STYLE

Layered monolith — controllers thin, services rich, entities never leave the service boundary

DTOs at every boundary — no entity is ever serialized to the client

@Transactional guards multi-step writes (status + history + stock)

DESIGN GOALS

Server-side authorization always — never trust the UI

Auditable — every status change is append-only

Deployable in 4 weeks, extensible after

HLD — Data Model & Lifecycle Overview

HLD-DATA

Entity overview

```
customers ──1:N── sites ──1:N── work_orders ──1:N── work_order_status_history (append-only)
                                      │  │
                                      │  └──1:N── part_usage ──N:1── parts
                                      └──1:N── time_logs ──N:1── users
```

Entity	Purpose
Customer / Site	Tenant org and its buildings — every work order anchors to both
WorkOrder	Central unit of work: priority, status, SLA due date, assignee
WorkOrderStatusHistory	Append-only audit trail of every transition
Part / PartUsage	Inventory catalog and transactional consumption per job
TimeLog	Technician labour minutes logged against a job

Work-order lifecycle (state machine)

NEW → assign → ASSIGNED → start → IN_PROGRESS → complete → COMPLETED → close → CLOSED

CANCELLED reachable from NEW/ASSIGNED (dispatcher/manager). ON_HOLD ↔ IN_PROGRESS (assigned technician only). CLOSED and CANCELLED are terminal. Full allow-list detailed in the LLD section, Page 5.

Role × action summary

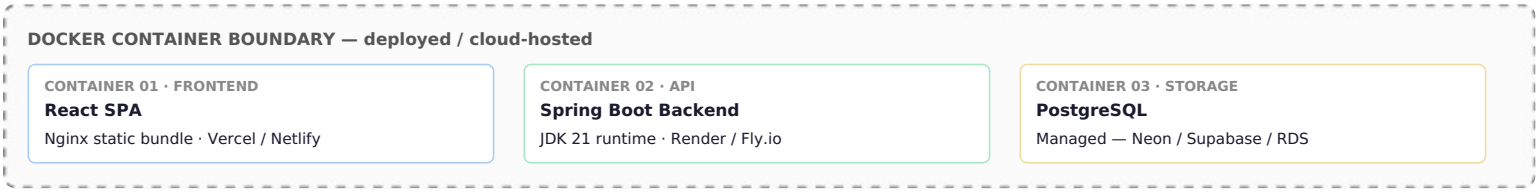
Action	Dispatcher	Technician	Manager	Customer
Create / assign work order	✓	—	✓	create own only
Start / hold / complete own job	—	✓	—	—
Close job	—	—	✓	—
View dashboard / all jobs	✓	own only	✓	own only

HLD — Technology Stack & Deployment

HLD-STACK

Layer	Technology	Why
Language	Java 21	Modern LTS — records, pattern matching
Backend	Spring Boot 3 (Web, Validation)	Industry-standard layered services
Security	Spring Security + JJWT	Stateless auth, method-level role enforcement
Persistence	Spring Data JPA / Hibernate	Entity mapping and repositories
Database	PostgreSQL 15+	Reliable relational store, ACID transactions
Migrations	Flyway	Versioned, reviewable schema changes
Frontend	React 18 + TypeScript (Vite)	Typed, component-based SPA
API docs	springdoc-openapi / Swagger UI	Live, browsable API reference

Deployment topology



Local dev: `docker-compose.yml` runs Postgres + backend together; frontend runs separately via `npm run dev` against `http://localhost:8080`.

Non-functional targets

Quality	Requirement
Security	Stateless JWT, BCrypt hashes, server-side role check on every protected action
Data integrity	Multi-step changes transactional; status history append-only; stock never negative
Performance	List endpoints paginated; no unbounded queries
Usability	Technician view works on a phone; dashboard answers a manager's key questions at a glance

PART B — LOW-LEVEL DESIGN (LLD)

Class/entity fields & methods, field-level schema, API contracts, sequence flow — the "how"

LLD — Entity & Service Class Design

LLD-CLASS

WorkOrder (JPA Entity)

id: Long · code: String (unique) · title: String · description: String · priority: Priority · status: WorkOrderStatus · customer: Customer · site: Site · assignedTo: User · createdBy: User · slaDueAt: Instant · slaBreached: boolean · createdAt / updatedAt: Instant

@PrePersist onCreate() — sets createdAt/updatedAt · @PreUpdate onUpdate() — refreshes updatedAt

WorkOrderService (business logic, @Transactional)

+ create(CreateWorkOrderRequest, actor): WorkOrder — computes slaDueAt from priority, writes initial NEW history row
+ assign(id, technicianId, actor): WorkOrder — NEW→ASSIGNED, dispatcher/manager only
+ transition(id, toStatus, note, actor): WorkOrder — validates against ALLOWED map, writes history row
+ logPartUsage(id, partId, qty, actor): WorkOrder — decrements Part.stockQty + inserts PartUsage, one transaction
+ logTime(id, minutes, note, actor): WorkOrder
- assertAllowedTransition(from, to) — throws IllegalStateTransitionException on invalid edge
- assertActorIsAssignee(workOrder, actorId) — throws ForbiddenException if not the assigned technician

JwtAuthFilter extends OncePerRequestFilter

doFilterInternal(request, response, chain) — extracts Bearer token, calls JwtUtil.parseClaims(), populates SecurityContext with AuthenticatedUser(id, email, role); clears context on JwtException

GlobalExceptionHandler (@RestControllerAdvice)

handleNotFound → 404 · handleForbidden → 403 · handleBadCredentials → 401 · handleIllegalTransition → 409 · handleInsufficientStock → 409 · handleValidation → 400

State-machine allow-list (implementation detail)

```
private static final Map<WorkOrderStatus, Set<WorkOrderStatus>> ALLOWED = Map.of(
    NEW,         Set.of(ASSIGNED, CANCELLED),
    ASSIGNED,    Set.of(IN_PROGRESS, CANCELLED),
    IN_PROGRESS, Set.of(ON_HOLD, COMPLETED),
    ON_HOLD,     Set.of(IN_PROGRESS),
    COMPLETED,   Set.of(CLOSED)
);
```

LLD — API Contracts & Field-Level Schema

LLD-API

PATCH /api/work-orders/{id}/status

```
// Request — StatusChangeRequest
{ "toStatus": "COMPLETED", "note": "Fixed compressor, tested airflow" }
   toStatus: enum, required   |   note: string, max 500, optional

// Response 200 — WorkOrderResponse
{
  "id": 42, "code": "WO-2026-00042", "status": "COMPLETED",
  "slaDueAt": "2026-08-28T18:00:00Z", "slaBreached": false,
  "history": [
    { "fromStatus": null, "toStatus": "NEW", "changedByName": "Dana Dispatcher", "changedAt": "..." },
    { "fromStatus": "IN_PROGRESS", "toStatus": "COMPLETED", "changedByName": "Theo Technician", "note": "...", "changedAt": "..." }
  ]
}

// Response 409 — illegal transition
{ "timestamp": "...", "status": 409, "message": "Cannot transition from COMPLETED to COMPLETED", "fieldErrors": [] }
```

POST /api/work-orders/{id}/parts

```
// Request — LogPartsRequest
{ "partId": 2, "qtyUsed": 1 }
   partId: Long, required   |   qtyUsed: int, min 1, required

// Service-layer guarantee: part_usage insert + parts.stock_qty decrement
// commit together inside one @Transactional method — see WorkOrderService.logPartUsage()
// Response 409 if stock_qty would go negative:
{ "status": 409, "message": "Insufficient stock for SKU ELEC-CAP-45UF: requested 3, available 1" }
```

Database — field-level detail (work_orders)

Column	Type	Constraint
code	VARCHAR(20)	UNIQUE NOT NULL — server-generated, e.g. WO-2026-00042
status	VARCHAR(20)	CHECK IN (NEW, ASSIGNED, IN_PROGRESS, ON_HOLD, COMPLETED, CLOSED, CANCELLED)
customer_id, site_id	BIGINT	NOT NULL FK — every work order always has both
sla_due_at	TIMESTAMPTZ	set at creation = created_at + offset(priority)

Indexes: `(status)`, `(assigned_to)`, `(customer_id)`, partial index `(sla_due_at) WHERE status NOT IN ('CLOSED','CANCELLED')` for the SLA scheduler scan.

LLD — Sequence Flow: Technician Completes a Job

LLD-SEQ

1. **Client → API** — PATCH /api/work-orders/42/status { toStatus: COMPLETED, note }, header Authorization: Bearer <jwt>
2. **JwtAuthFilter** — parses claims, populates SecurityContext(userId=17, role=TECHNICIAN); rejects with 401 if expired/tampered
3. **WorkOrderController.transitionStatus()** — @PreAuthorize("hasAnyRole('TECHNICIAN','DISPATCHER','MANAGER')"); Bean Validation on StatusChangeRequest
4. **WorkOrderService.transition(42, COMPLETED, note, actor=17)** — loads WorkOrder via repository
5. **assertActorIsAssignee()** — checks actor.id == workOrder.assignedTo.id; throws ForbiddenException (403) otherwise
6. **assertAllowedTransition(IN_PROGRESS, COMPLETED)** — checks the ALLOWED map; throws IllegalStateTransitionException (409) if not present
7. **Persist** — workOrder.status = COMPLETED, save(); insert WorkOrderStatusHistory(from=IN_PROGRESS, to=COMPLETED, by=17, note) — same @Transactional method, both commit together
8. **Controller → Client** — 200 OK, WorkOrderResponse DTO serialized (entity never leaves the service layer)

Exception → HTTP mapping (GlobalExceptionHandler)

Exception	HTTP	When it fires in this flow
BadCredentialsException	401	Token missing, expired, or tampered
ForbiddenException	403	Actor is not the assigned technician
IllegalStateTransitionException	409	Requested (from, to) pair not in ALLOWED map
MethodArgumentNotValidException	400	toStatus missing, note over 500 chars

Every step above runs inside a single request/response cycle. Steps 4–7 are inside one `@Transactional` service method — if step 7's history insert failed, the status update in step 7 would roll back too, keeping `work_orders.status` and `work_order_status_history` always consistent.